

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«КРОС-ПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри АІС Бельська В. Ю.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 1

Тема: Основні керуючі оператори мови java. Масиви.

Мета: отримати навички у створенні консольних додатків з використанням основних керуючих операторів та масивів.

Порядок виконання роботи:

1. Вантажний літак повинен пролетіти з вантажем з пункту А пункт С через пункт В. Ємність бака для палива у літака – Х літрів (значення зчитується з файлу). Споживання палива на 1 км залежно від ваги вантажу літака таке:

- до 500 кг - 1 літрів/км;
- до 1000 кг - 4 літрів/км;
- до 1500 кг - 7 літрів/км;
- до 2000 кг - 9 літрів/км;
- більше 2000 кг – літак не піднімає.

З файлу вводиться відстань між пунктами А і В і відстань між пунктами В і С, а також вага вантажу. Програма повинна розрахувати, яку мінімальну кількість палива необхідно для дозаправки літака в пункті В, щоб долетіти з пункту А до пункту С. У разі неможливості подолати будь-яку з відстаней – програма повинна вивести повідомлення про неможливість польоту за введеним маршрутом.

3. Створіть двовимірний масив довільної розмірності. Знайдіть найбільший елемент масиву. Перемістіть знайдений елемент у лівий верхній кут масиву шляхом перестановки рядків та стовпчиків масиву.

5. Гра 2048 – це логічна головоломка, де на полі 4×4 потрібно переміщувати клітинки ігрового поля з числами (ступенями двійки), об'єднуючи клітинки з однаковими числами для отримання клітинки з номіналом «2048».

Основна мета – об'єднувати клітинки, що розташовані поряд, сумуючи їх значення ($2+2=4$, $4+4=8$, $8+8=16...$), поки не буде досягнуто заповітної цифри. за наступними правилами:

Основні правила та механіка:

- Початок гри: на ігровій матриці розмірності 4×4 лише 2 довільні клітинки заповнені цифрою «2» - усі інші порожні (0 на екран не виводити)

- Управління: використовуються стрілки на клавіатурі щоб рухати усі клітинки одночасно в один із чотирьох боків: вгору, вниз, вліво або вправо.
 - Об'єднання: коли дві клітинки з однаковим числом стикаються при русі, вони зливаються в одну, сума якої дорівнює їхньому загальному значенню.
 - Нові значення на полі: після кожного ходу одне нульове значення у масиві автоматично замінюється «2» (90% ймовірності) або «4» (10% ймовірності).
 - Програш: гра закінчується, якщо поле(усі комірки багатовимірною масиву) повністю заповнене і неможливо зробити хід (немає сусідніх однакових плиток для об'єднання).
 - Перемога: Ви досягаєте мети, коли отримуєте плитку з номіналом 2048, але можна продовжувати гру далі для встановлення рекорду.
 - Нарахування балів (score): на початку гри гравець має 0 балів. Впродовж гри бали збільшуються на величину числа, отриманого у результаті об'єднання клітинок, що мають одне значення.
- Забезпечити гравцю можливість перервати гру, розпочати нову гру (багаторазове використання гри). На протязі використання програмного коду одним гравцем запам'ятовувати його найкращі результати з попередніх ігор при багаторазовому варіанті гри.

1.

```
// Підключення класів для роботи з файлами
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

// Головний клас програми
public class PlaneFlight {

    public static void main(String[] args) {

        try {

            // Створення об'єкта Scanner для зчитування даних з файлу
            Scanner input = new Scanner(new FileReader("data.txt"));

            // Зчитування ємності бака літака
```

```
double tank = input.nextDouble();

// Зчитування відстані між пунктами A і B
double distanceAB = input.nextDouble();

// Зчитування відстані між пунктами B і C
double distanceBC = input.nextDouble();

// Зчитування ваги вантажу
double weight = input.nextDouble();

// Змінна для витрати палива на 1 км
double fuelPerKm = 0;

// Визначення витрати палива залежно від ваги
if (weight <= 500) {
    fuelPerKm = 1;
} else if (weight <= 1000) {
    fuelPerKm = 4;
} else if (weight <= 1500) {
    fuelPerKm = 7;
} else if (weight <= 2000) {
    fuelPerKm = 9;
} else {
    // Якщо вантаж більше 2000 кг - літак не може злетіти
    System.out.println("Літак не може підняти такий вантаж.");
    return;
}

// Розрахунок необхідного палива для ділянки A-B
double fuelAB = distanceAB * fuelPerKm;

// Розрахунок необхідного палива для ділянки B-C
double fuelBC = distanceBC * fuelPerKm;

// Перевірка чи може літак долетіти до пункту B
if (fuelAB > tank) {
    System.out.println("Неможливо долетіти з A до B.");
    return;
}

// Перевірка чи можливо подолати відстань B-C
if (fuelBC > tank) {
    System.out.println("Неможливо долетіти з B до C навіть після заправки.");
    return;
}

// Кількість палива, що залишиться після польоту A-B
double fuelLeft = tank - fuelAB;

// Мінімальна кількість палива для дозаправки
```

```

        double refuel = fuelBC - fuelLeft;

        // Якщо палива вистачає без дозаправки
        if (refuel < 0) {
            refuel = 0;
        }

        // Виведення результату
        System.out.println("Мінімальна кількість палива для дозаправки: " +
refuel + " літрів");

    } catch (IOException ex) {

        // Повідомлення про помилку читання файлу
        System.out.println("Помилка читання файлу: " + ex.getMessage());
    }
}
}

```

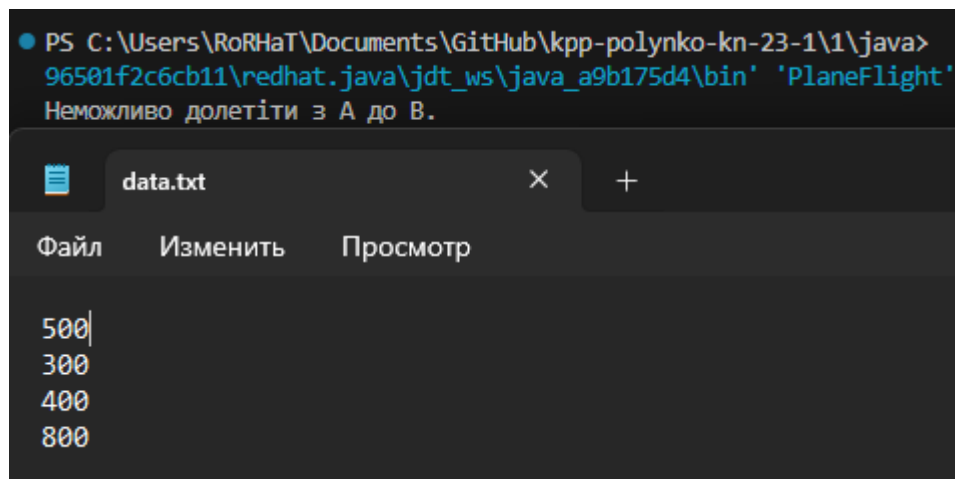


Рисунок 1.1 – Результат роботи скрипту завдання 1

3.

```

import java.util.Random;

public class MatrixTask {

    public static void main(String[] args) {

        // Розмірність масиву
        int rows = 4;
        int cols = 5;

        // Створення двовимірної масиву
        int[][] matrix = new int[rows][cols];
    }
}

```

```

Random rand = new Random();

// Заповнення масиву випадковими числами
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = rand.nextInt(100); // числа від 0 до 99
    }
}

System.out.println("Початковий масив:");
printMatrix(matrix);

// Пошук максимального елемента
int max = matrix[0][0];
int maxRow = 0;
int maxCol = 0;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {

        if (matrix[i][j] > max) {
            max = matrix[i][j];
            maxRow = i;
            maxCol = j;
        }
    }
}

System.out.println("Максимальний елемент: " + max);

// Перестановка рядків (мінємо рядок з максимальним елементом з
нульовим)
int[] tempRow = matrix[0];
matrix[0] = matrix[maxRow];
matrix[maxRow] = tempRow;

// Перестановка стовпців
for (int i = 0; i < rows; i++) {

    int temp = matrix[i][0];
    matrix[i][0] = matrix[i][maxCol];
    matrix[i][maxCol] = temp;
}

System.out.println("Масив після перестановки:");
printMatrix(matrix);
}

// Метод виводу масиву
public static void printMatrix(int[][] m) {

    for (int i = 0; i < m.length; i++) {

```

```

        for (int j = 0; j < m[i].length; j++) {
            System.out.print(m[i][j] + "\t");
        }

        System.out.println();
    }
}

```

● Початковий масив:

46	86	56	91	92
99	57	60	7	61
16	92	2	43	4
63	24	70	46	38

Максимальний елемент: 99

Масив після перестановки:

99	57	60	7	61
46	86	56	91	92
16	92	2	43	4
63	24	70	46	38

Рисунок 1.2 – Результат роботи скрипту завдання 3

5.

```

import java.util.Random;
import java.util.Scanner;

public class Game2048 {

    static int[][] field = new int[4][4];
    static int score = 0;
    static int bestScore = 0;
    static Random random = new Random();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean play = true;

        while (play) {

            initGame();

            while (true) {

                printField();

                if (!canMove()) {
                    System.out.println("Гра завершена! Нема можливих ходів.");
                }
            }
        }
    }
}

```

```

        break;
    }

    System.out.println("Введіть хід (W-вгору, S-вниз, A-вліво, D-
вправо, Q-вихід):");
    char move = sc.next().toUpperCase().charAt(0);

    boolean moved = false;

    if (move == 'A')
        moved = moveLeft();
    if (move == 'D')
        moved = moveRight();
    if (move == 'W')
        moved = moveUp();
    if (move == 'S')
        moved = moveDown();
    if (move == 'Q')
        return;

    if (moved) {
        addRandomNumber();
    }
}

if (score > bestScore) {
    bestScore = score;
}

System.out.println("Ваш рахунок: " + score);
System.out.println("Найкращий рахунок: " + bestScore);
System.out.println("Почати нову гру? (Y/N)");

if (!sc.next().equalsIgnoreCase("Y")) {
    play = false;
}
}

static void initGame() {
    field = new int[4][4];
    score = 0;
    addRandomNumber();
    addRandomNumber();
}

static void printField() {

    System.out.println("\nРахунок: " + score + "    Найкращий: " + bestScore);

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

```



```

        if (field[i][j] == 0)
            System.out.print(".\t");
        else
            System.out.print(field[i][j] + "\t");
    }
    System.out.println();
}

static void addRandomNumber() {

    int emptyCount = 0;

    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            if (field[i][j] == 0)
                emptyCount++;

    if (emptyCount == 0)
        return;

    int position = random.nextInt(emptyCount);
    int count = 0;

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

            if (field[i][j] == 0) {

                if (count == position) {
                    field[i][j] = (random.nextInt(10) < 9) ? 2 : 4;
                    return;
                }
                count++;
            }
        }
    }
}

static boolean moveLeft() {

    boolean moved = false;

    for (int i = 0; i < 4; i++) {

        for (int j = 1; j < 4; j++) {

            if (field[i][j] != 0) {

                int col = j;

```

```

        while (col > 0 && field[i][col - 1] == 0) {
            field[i][col - 1] = field[i][col];
            field[i][col] = 0;
            col--;
            moved = true;
        }

        if (col > 0 && field[i][col - 1] == field[i][col]) {
            field[i][col - 1] *= 2;
            score += field[i][col - 1];
            field[i][col] = 0;
            moved = true;
        }
    }
}

return moved;
}

static boolean moveRight() {

    rotateHorizontal();
    boolean moved = moveLeft();
    rotateHorizontal();

    return moved;
}

static boolean moveUp() {

    transpose();
    boolean moved = moveLeft();
    transpose();

    return moved;
}

static boolean moveDown() {

    transpose();
    rotateHorizontal();
    boolean moved = moveLeft();
    rotateHorizontal();
    transpose();

    return moved;
}

static void rotateHorizontal() {

    for (int i = 0; i < 4; i++) {

```

```

        for (int j = 0; j < 2; j++) {

            int temp = field[i][j];
            field[i][j] = field[i][3 - j];
            field[i][3 - j] = temp;

        }
    }
}

static void transpose() {

    for (int i = 0; i < 4; i++) {
        for (int j = i + 1; j < 4; j++) {

            int temp = field[i][j];
            field[i][j] = field[j][i];
            field[j][i] = temp;

        }
    }
}

static boolean canMove() {

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

            if (field[i][j] == 0)
                return true;

            if (j < 3 && field[i][j] == field[i][j + 1])
                return true;

            if (i < 3 && field[i][j] == field[i + 1][j])
                return true;

        }
    }

    return false;
}
}

```

```

Рахунок: 52    Найкращий: 0
2      .      .      2
.      .      .      .
.      .      .      16
.      .      4      2
Введіть хід (W-вгору, S-вниз, A-вліво, D-вправо, Q-вихід):

```

Рисунок 1.3 – Результат роботи скрипту завдання 5

Висновок: на цій лабораторній роботі ми отримали навички у створення консольних додатків з використанням основних керуючих операторів та масивів, реалізували гру три додатка на мові java.